Bubble sort

```
import time

# Function for Selection Sort
def selection_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        min_index = i

        # Find the minimum element in the remaining unsorted array
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        # Swap the minimum element with the current element
        arr[i], arr[min_index] = arr[min_index], arr[i]

# User Input
n = int(input("Enter the number of elements: "))

arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Start Timer
start_time = time.perf_counter()

# Perform Selection Sort
selection_sort(arr)

# End Timer
end_time = time.perf_counter()

# Calculate Execution Time
execution_time = end_time - start_time

# Display Results
print("\nSorted Array:")
print(arr)

print(f"\nExecution Time: {execution_time:.8f} seconds")

print("\nTime Complexity:")
print("Best Case    : O(n²)")
print("Average Case : O(n²)")
print("Worst Case   : O(n²)")

print("\nSpace Complexity: O(1)")6
```

```
Enter the number of elements: 5
Enter the elements:
3
4
5
7
8
```

```
Sorted Array:
[3, 4, 5, 7, 8]

Execution Time: 0.00008939 seconds

Time Complexity:
Best Case    : O(n²)
Average Case : O(n²)
Worst Case   : O(n²)

Space Complexity: O(1)
```

Selection sort

```
import time

# Function for Bubble Sort
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        # Traverse through all array elements
        for j in range(0, n - i - 1):
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

# User Input
n = int(input("Enter the number of elements: "))

arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Start Timer
start_time = time.perf_counter()

# Perform Bubble Sort
bubble_sort(arr)

# End Timer
end_time = time.perf_counter()

# Calculate Execution Time
execution_time = end_time - start_time

# Display Results
print("\nSorted Array:")
print(arr)

print(f"\nExecution Time: {execution_time:.8f} seconds")

print("\nTime Complexity:")
print("Best Case    : O(n)")
print("Average Case : O(n²)")
print("Worst Case   : O(n²)")
```

```
print("\nSpace Complexity: O(1)")
```

```
Enter the number of elements: 5
Enter the elements:
45
35
89
9
3

Sorted Array:
[3, 9, 35, 45, 89]

Execution Time: 0.00009930 seconds

Time Complexity:
Best Case    : O(n)
Average Case : O(n²)
Worst Case   : O(n²)

Space Complexity: O(1)
```

Insertion sort

```
import time

# Function for Insertion Sort
def insertion_sort(arr):
    n = len(arr)

    for i in range(1, n):
        key = arr[i]
        j = i - 1

        # Move elements greater than key one position ahead
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

# User Input
n = int(input("Enter the number of elements: "))

arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Start Timer
start_time = time.perf_counter()

# Perform Insertion Sort
insertion_sort(arr)

# End Timer
end_time = time.perf_counter()

# Calculate Execution Time
execution_time = end_time - start_time
```

```
# Display Results
print("\nSorted Array:")
print(arr)

print(f"\nExecution Time: {execution_time:.8f} seconds")

print("\nTime Complexity:")
print("Best Case    : O(n)")
print("Average Case : O(n²)")
print("Worst Case   : O(n²)")

print("\nSpace Complexity: O(1)")
```

```
Enter the number of elements: 5
Enter the elements:
45
78
2
3
5

Sorted Array:
[2, 3, 5, 45, 78]

Execution Time: 0.00008753 seconds

Time Complexity:
Best Case    : O(n)
Average Case : O(n²)
Worst Case   : O(n²)

Space Complexity: O(1)
```

Merge sort

```
import time

# Function for Merge Sort
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2

        # Divide the array into two halves
        left = arr[:mid]
        right = arr[mid:]

        # Recursively sort both halves
        merge_sort(left)
        merge_sort(right)

        i = j = k = 0

        # Merge the two sorted halves
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
```

```
            k += 1

        # Copy remaining elements of left half
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1

        # Copy remaining elements of right half
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1

# User Input
n = int(input("Enter the number of elements: "))

arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Start Timer
start_time = time.perf_counter()

# Perform Merge Sort
merge_sort(arr)

# End Timer
end_time = time.perf_counter()

# Calculate Execution Time
execution_time = end_time - start_time

# Display Results
print("\nSorted Array:")
print(arr)

print(f"\nExecution Time: {execution_time:.8f} seconds")

print("\nTime Complexity:")
print("Best Case    : O(n log n)")
print("Average Case : O(n log n)")
print("Worst Case   : O(n log n)")

print("\nSpace Complexity: O(n)")
```

```
Enter the number of elements: 5
Enter the elements:
56
3
67
89
34

Sorted Array:
[3, 34, 56, 67, 89]

Execution Time: 0.00020735 seconds

Time Complexity:
Best Case    : O(n log n)
```

```
Average Case : O(n log n)
Worst Case   : O(n log n)

Space Complexity: O(n)
```

Quick sort

```
import time

# Function to partition the array
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1

    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]

    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

# Function for Quick Sort
def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)

        # Sort elements before and after partition
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

# User Input
n = int(input("Enter the number of elements: "))

arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Start Timer
start_time = time.perf_counter()

# Perform Quick Sort
quick_sort(arr, 0, n - 1)

# End Timer
end_time = time.perf_counter()

# Calculate Execution Time
execution_time = end_time - start_time

# Display Results
print("\nSorted Array:")
print(arr)

print(f"\nExecution Time: {execution_time:.8f} seconds")

print("\nTime Complexity:")
print("Best Case    : O(n log n)")
print("Average Case : O(n log n)")
```

```
print("Worst Case   : O(n²)")

print("\nSpace Complexity: O(log n)")63
```

```
Enter the number of elements: 5
Enter the elements:
45
7
2
4
8

Sorted Array:
[2, 4, 7, 8, 45]

Execution Time: 0.00020809 seconds

Time Complexity:
Best Case    : O(n log n)
Average Case : O(n log n)
Worst Case   : O(n²)

Space Complexity: O(log n)
```